

Model Training and Algorithms (Complete)

This document explains how the fake-news model is built, trained, evaluated, and used in the full decision pipeline.

1. What Model Is Used

Primary ML classifier:

- Model family: Transformer (DistilBERT)
- Checkpoint used for fine-tuning: `distilbert-base-multilingual-cased`
- Task: Binary sequence classification
- Labels:
 - `0` = REAL
 - `1` = FAKE

Saved artifact config (`backend/model/fake_news_model/config.json`):

- Architecture: `DistilBertForSequenceClassification`
- Layers: 6
- Hidden size: 768
- Attention heads: 12
- Max position embeddings: 512
- Vocab size: 119547

2. Algorithms Used in the System

The project uses a multi-layer detection architecture, not only one algorithm.

1. Supervised Deep Learning Classifier (Primary)

- Algorithm: DistilBERT fine-tuning for binary classification
- Library stack: PyTorch + Hugging Face Transformers
- Script: `backend/scripts/train_model.py`

2. Knowledge/Rule Verification

- Algorithm type: Rule-based pattern matching (regex + domain rules)
- Script: `backend/services/knowledge_verification_service.py`
- Purpose: Detect factually impossible or highly suspicious claims.

3. Real-Time Verification

- Algorithm type: Heuristic + API cross-checking
- Script: `backend/services/verification_service.py`
- Purpose: Validate claims against external data signals (e.g., finance/crypto/news checks).

4. LLM-Based Verification

- Algorithm type: Prompted LLM judgment with structured JSON output
- Script: `backend/services/openai_verification_service.py`
- Providers supported: Gemini, OpenAI, Ollama (with fallback behavior).

5. Fusion Decision Engine

- Algorithm type: Weighted evidence fusion (score aggregation + overrides)
- Script: `backend/services/fusion_service.py`
- Purpose: Combine ML + rules + APIs + LLM + source credibility into one final verdict.

3. Training Data Sources

Data preparation script:

- `backend/scripts/prepare_dataset.py`

Raw sources used:

- ISOT (`Fake.csv`, `True.csv`)
- FakeNewsNet (`gossipcop_*`, `politifact_*`)
- WELFake (`WELFake_Dataset.csv`)
- LIAR (`train.tsv`, `valid.tsv`, `test.tsv`)

Label mapping rules:

- Global training labels:
 - REAL -> `0`
 - FAKE -> `1`
- LIAR mapping:
 - `pants-fire`, `false`, `barely-true` -> `1` (fake)
 - `half-true`, `mostly-true`, `true` -> `0` (real)

4. Data Cleaning and Preprocessing

Implemented in `prepare_dataset.py`:

- Missing text/label rows removed
- Text whitespace normalized
- Labels forced to numeric and filtered to `{0, 1}`
- Length filtering:
 - LIAR rows: min 10 chars
 - Other datasets: min 50 chars
- Duplicate rows removed by text

Text construction logic:

- When both `title` and `text` exist:
 - If `text` is empty, fallback to `title`
 - Else concatenate `title + text`

5. Train/Validation/Test Split Strategy

Split algorithm:

- Stratified manual split by label class
- Ratio: 70% train / 15% val / 15% test
- Reproducibility seed: default 42

Current prepared dataset stats (`backend/training/processed/dataset_stats.json`):

- Total rows: 94,439
- Train rows: 66,107
- Val rows: 14,166
- Test rows: 14,166
- Full label counts:
 - REAL (0): 55,653
 - FAKE (1): 38,786

6. Tokenization and Input Encoding

Tokenizer:

- `DistilBertTokenizerFast`
- Same checkpoint family as model

Encoding settings:

- `max_length = 512`
- `padding = max_length`
- `truncation = True`
- Returns PyTorch tensors

7. Training Configuration (Actual)

From `backend/scripts/train_model.py` and saved metrics:

- Epochs: 3
- Batch size: 16
- Learning rate: $2e-5$
- Weight decay: 0.01
- Best model selection metric: f1
- Evaluation strategy: every epoch
- Save strategy: every epoch
- Logging steps: every 100 steps
- Save total limit: 2 checkpoints
- Mixed precision:
 - `fp16 = True` when CUDA is available
 - `bf16 = False`
- Gradient checkpointing: enabled when CUDA is available

8. Evaluation Metrics and Formulas

Metrics computed:

- Accuracy
- Precision
- Recall
- F1-score

Confusion matrix terms:

- TP, TN, FP, FN

Formulas:

- $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$
- $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$
- $\text{F1} = 2 * \text{Precision} * \text{Recall} / (\text{Precision} + \text{Recall})$
- $\text{Accuracy} = (\text{TP} + \text{TN}) / \text{N}$

9. Latest Recorded Performance

From `backend/model/training_metrics.json`:

Validation:

- Accuracy: **0.9171**
- Precision: **0.9115**
- Recall: **0.8838**
- F1: **0.8975**
- Loss: **0.2435**

Test:

- Accuracy: **0.9153**
- Precision: **0.9114**
- Recall: **0.8792**
- F1: **0.8950**
- Loss: **0.2481**

Training runtime snapshot:

- Device: **NVIDIA GeForce RTX 2050**
- Runtime: **7676.04 sec** (~2.13 hours)
- Train samples/sec: **25.836**
- Train steps/sec: **1.615**

10. How Inference Works in Production

Inference service:

- `backend/services/fake_news_service.py`

Flow:

1. Load model + tokenizer once at startup.
2. Detect language (`langdetect`).
3. For non-English text, attempt translation to English (`deep_translator`).
4. Run model forward pass.
5. Apply softmax to logits.
6. Return:
 - predicted class (`REAL/FAKE`)
 - confidence
 - class probabilities

11. Fusion Layer Decision Logic

Fusion service (`fusion_service.py`) combines evidence from:

- ML classifier
- Knowledge verification
- Real-time API checks
- LLM verification
- Source credibility

Core algorithm:

- Each evidence item has:
 - conclusion (`real` or `fake`)
 - confidence
 - weight
- Weighted fake and real scores are aggregated and normalized.
- Final class = higher normalized score.
- Override rules can force decision for high-severity evidence.

Example weight concepts in code:

- Knowledge high severity: `1.0`
- Real-time API verified inconsistency: `0.9`
- ML high confidence: `0.7`
- LLM high confidence: `0.75`

12. Commands to Reproduce Training

From project root (Windows):

```
# 1) Prepare dataset
.\backend\venv310\Scripts\python.exe backend\scripts\prepare_dataset.py

# 2) Train model
.\backend\venv310\Scripts\python.exe backend\scripts\train_model.py

# 3) Evaluate model
.\backend\venv310\Scripts\python.exe backend\scripts\evaluate_model.py
```

Outputs generated:

- `backend/training/processed/*.csv`
- `backend/training/processed/dataset_stats.json`
- `backend/model/fake_news_model/*`
- `backend/model/tokenizer/*`
- `backend/model/training_metrics.json`
- `backend/model/evaluation_metrics.json`

13. Dependency Files for Training

- Runtime/API dependencies: `backend/requirements.txt`
- Minimal training extras: `backend/requirements-training.txt`

`requirements-training.txt` includes:

- `numpy`
- `pandas`
- `torch`
- `transformers`
- `accelerate`
- `safetensors`
- `sentencepiece`

14. Current Limitations and Notes

- Model is binary (`REAL` vs `FAKE`), no multi-class stance/explanation labels.
- Training uses max sequence length 512, so very long texts are truncated.
- Label quality depends on merged external datasets and mapping assumptions.
- LLM/rule/API layers improve robustness but are still heuristic and configuration-dependent.

If you retrain with new hyperparameters/datasets, regenerate this file sections 5, 7, and 9 from updated `dataset_stats.json` and `training_metrics.json`.